Team Mittens (L17)

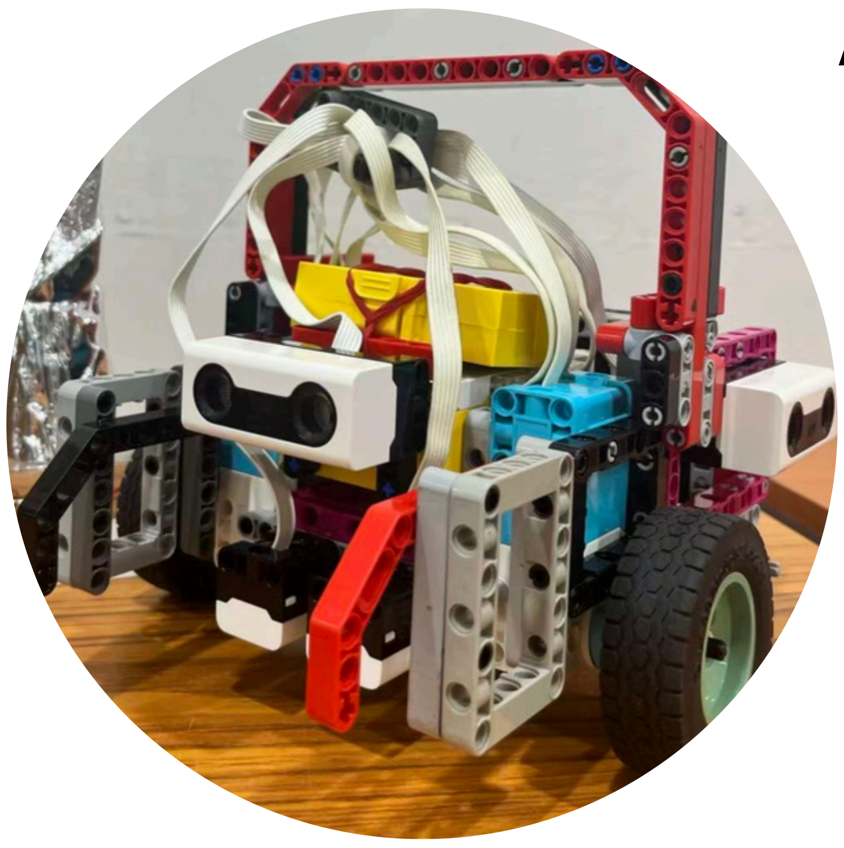
Team members:

Wang Ziqing - team leader and lead builder

- responsible for all mechanical design and structural decisions. She continuously thought of new ways to improve the robot, iterating on the chassis, stabilizer arms, and modular weight system to maximize performance and adaptability.

Zhong Zhangyuan - software head

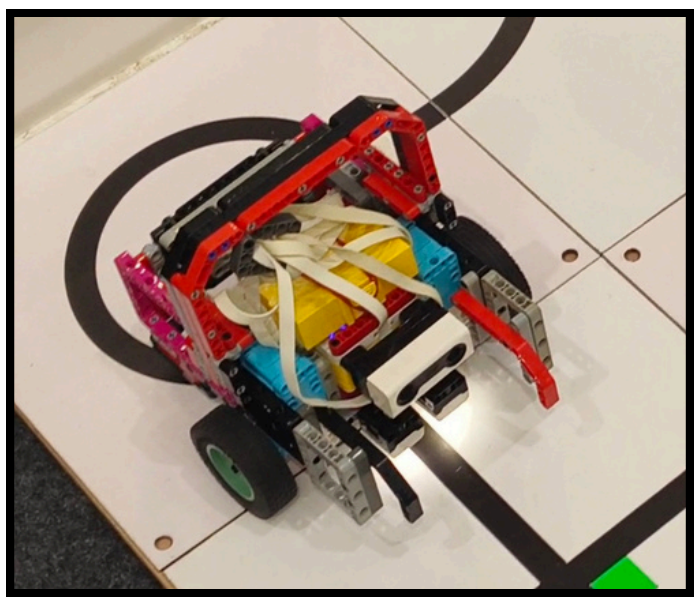
- leads all programming and software development, designing and implementing the algorithms behind the robot's navigation and obstacle handling. His rigorous testing approach ensured consistent and reliable robot performance across all competition scenarios.



About our robot:

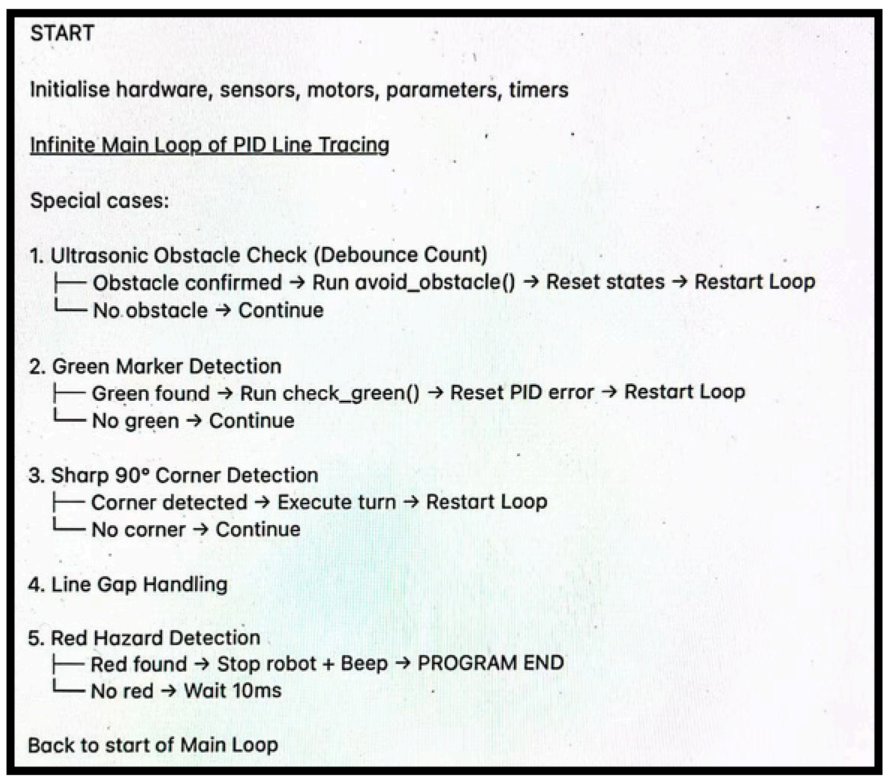
- Our robot is a compact, highly adaptable rescue platform built on a robust LEGO SPIKE Prime chassis, designed to handle the full range of challenges present in the RoboCup Junior Rescue competition.
- Its most distinctive feature is its modular, reconfigurable architecture — components can be added, removed, or repositioned on-the-fly to dynamically shift the centre of gravity depending on track conditions. The wide wheel stance and front stabiliser arms allow the robot to traverse speed bumps and descend ramps smoothly without tipping, while high-grip drive wheels and slim blade-profiled rear elements maximise traction and enable tight, precise turns.
- On the software side, the robot runs a PID control algorithm for accurate and consistent line following, paired with a distance sensor for reliable obstacle detection and avoidance. The codebase has been extensively tested and proven to perform stably and consistently across the vast majority of runs attempted, reflecting the team's rigorous approach to software reliability. Together, these hardware and software design choices produce a robot that balances stability, manoeuvrability, and adaptability.

Hardware:



Our robot is a compact, highly adaptable rescue platform built on a robust LEGO-based chassis, designed to handle the full range of challenges present. The robot's wide wheel and support stance, both front and rear, allows it to traverse obstacles such as speed bumps with ease, as the chassis width exceeds that of most common obstacles. At the front, two purpose-built stabilizer arms act as ground props, enabling the robot to smoothly descend ramps and self-stabilize upon landing without tipping. The base structure is engineered for high structural integrity, ensuring consistent sensor alignment and mechanical reliability across runs. High-grip wheels are employed on the drive axles to maximize traction on varied surfaces, while the rear of the robot uses slim, blade-profiled LEGO elements that minimize ground contact area, significantly reducing friction during turns and enabling tight, precise rotations. Together, these design choices produce a robot that balances stability, maneuverability, and adaptability.

Software:



START

Initialise hardware, sensors, motors, parameters, timers

Infinite Main Loop of PID Line Tracing

Special cases:

1. Ultrasonic Obstacle Check (Debounce Count)
   ├── Obstacle confirmed → Run avoid_obstacle() → Reset states → Restart Loop
   └── No obstacle → Continue

2. Green Marker Detection
   ├── Green found → Run check_green() → Reset PID error → Restart Loop
   └── No green → Continue

3. Sharp 90° Corner Detection
   ├── Corner detected → Execute turn → Restart Loop
   └── No corner → Continue

4. Line Gap Handling

5. Red Hazard Detection
   ├── Red found → Stop robot + Beep → PROGRAM END
   └── No red → Wait 10ms

Back to start of Main Loop

This Pybricks program uses PID control for core line tracking with reflection values from two color sensors. The main loop runs continuously with fixed task priorities: filtered ultrasonic obstacle detection triggers preset avoidance maneuvers. It detects green markers via HSV thresholds, then executes timed approach, verification and dedicated turning sequences. Sharp corners are handled with direct 90° turns. Logic for line gaps keeps the robot moving straight with a single minor correction. Red color detection halts all movement, sounds a warning beep and exits the program. Multiple timers and state variables manage motion phases, filter false sensor readings and track operational states throughout autonomous runs.